# ConectaCampus

Arquitetura distribuída

PJBL | Entrega 3 | 26 de outubro de 2026

## ARC42, C4, Architecture Canvas e evidências

Implementação publicada no GitHub. A conclusão da entrega depende de implantação cloud, imagens Docker Hub, evidências públicas e vídeo do grupo.

| Validação | Estado |
| --- | --- |
| Testes locais e componentes | Aprovados |
| Interface desktop e móvel | Aprovada em três testes de navegador |
| Compilação React e Bicep | Aprovadas |
| Auditoria de dependências | Sem alertas |
| Integração real | Aprovada no GitHub Actions: MongoDB, SQL Server, Azurite, Functions Host, outbox e ledger |
| Cloud, Docker Hub e vídeo | Pendentes |

## Equipe

Anna Quezia Dos Santos, Laura Rodrigues de Souza de Camargo, Leonardo Smijtink, Otávio Augusto Fermino da Silva

Projeto PJBL, Entrega 3. Implementação e preparação da demonstração.

# 1. Introdução e objetivos

O ConectaCampus reúne eventos acadêmicos e certificados. O objetivo técnico é demonstrar microfrontend React, BFF Node.js, dois microserviços, bancos distintos, gateway e Azure Functions. A apresentação deve usar URLs públicas e fala de todos os alunos.

Alunos: Anna Quezia Dos Santos, Laura Rodrigues de Souza de Camargo, Leonardo Smijtink e Otávio Augusto Fermino da Silva. Nomes preservados do GRUPO.md original.

# 2. Restrições

O frontend consome somente o BFF pelo gateway. Eventos pertencem ao MongoDB e certificados ao Azure SQL. Nenhum serviço acessa o banco do outro. Código público, READMEs, Swagger, Docker e testes são obrigatórios. A apresentação local não substitui a demonstração cloud.

A referência “Free 1 DTU” do enunciado difere da oferta gratuita atual do Azure SQL, baseada em cotas serverless. Confira a elegibilidade da assinatura. Não foram criados recursos cloud nem assumidos custos nesta etapa.

# 3. Contexto e escopo

Estudantes e organizadores consultam eventos e administram certificados. MongoDB Atlas, Azure SQL e Storage são dependências externas. Não há integração com sistemas acadêmicos institucionais ou autenticação por aluno nesta demonstração. Use dados fictícios.

Os modelos C4 de contexto, containers e componentes estão em docs/c4.md e nas páginas gráficas do PDF. O Architecture Canvas está em docs/canvas.md.

# 4. Estratégia da solução

A SPA possui módulos de Resumo, Agenda e Certificados, carregados separadamente. Nginx centraliza a entrada. O BFF faz proxy dos CRUDs e agrega dados em GET /aggregated-data. Foram preservados os domínios do projeto original.

Clean Architecture direciona as dependências ao domínio. Na Application dos microserviços, cada operação CRUD tem seu slice. Infrastructure implementa repositórios, enquanto API expõe HTTP e Swagger.

# 5. Visão de blocos

O BFF consulta eventos e certificados em paralelo, calcula contagens e chama a Function HTTP. A resposta contém eventos, certificados, indicadores e generatedAt. O BFF não acessa bancos diretamente.

Eventos valida capacidade e normaliza lotação. Certificados valida carga horária de 1 a 1000 horas, data real e campos obrigatórios. Eventos usa ObjectId e certificados usa UUID. Drivers recebem campos permitidos e parâmetros.

A Function registra enrich e events por HTTP e campusEvents por Queue Trigger. O ledger no Table Storage usa id de evento como chave única. O contrato v1 não contém o nome do participante.

## 6. Visão de execução

Consulta: navegador, gateway, BFF, dois serviços e Function HTTP. Uma falha de dependência gera 502 ou 504. Um conjunto incompleto não é informado como sucesso completo.

Escrita: após validar, o SqlRepository grava certificado e outbox em uma transação. O publicador envia pendências e confirma publishedAt somente após envio. Uma falha entre envio e confirmação pode gerar reentrega.

Consumo: o Queue Trigger valida o contrato e cria a entidade no ledger. Chave existente significa duplicata. Mensagens inválidas vão para campus-events-poison após cinco falhas. O fluxo é eventualmente consistente e tem entrega pelo menos uma vez.

## 7. Visão de implantação

Compose contém gateway, frontend, BFF, serviços, MongoDB, SQL Server, Azurite e Azure Functions Host. Apenas a porta 8080 é publicada. O banco local e o schema SQL são inicializados. Credenciais ficam fora do Git.

Bicep define Container Apps com gateway público e demais apps internos. Atlas, Azure SQL, Storage e Function App são pré-requisitos descritos no guia de deploy. O ingress Azure termina TLS. Os cinco componentes têm Dockerfiles e repositórios independentes. Gateway e infraestrutura ficam no repositório de integração.

## 8. Conceitos transversais

Segurança: token interno, Function key no servidor, limites de requisições, payload e métodos, bancos sem portas publicadas. Não há login de usuário final. A demonstração deve conter somente dados de teste.

Observabilidade: X-Request-ID no BFF, logs de falha da outbox e id do evento processado. /events permite comparar publicação e ledger. Health informa processo ativo. Smoke verifica dependências e dados.

Testes: regras puras, CRUD HTTP, fronteiras de imports, interface em navegador e layout móvel. O pipeline de integração usa drivers e infraestrutura reais no Compose. A prévia local usa memória apenas de forma explícita.

## 9. Decisões de arquitetura

ADR 01: preservar eventos, certificados e a interface do ConectaCampus. ADR 02: manter bancos isolados por domínio. ADR 03: usar outbox para persistência atômica e consumidor idempotente para reentrega.

ADR 04: usar Azure Queue e Table Storage para mensageria e evidência de processamento. ADR 05: usar Nginx como gateway e TLS do Container Apps. AWS Gateway era uma opção tecnológica do enunciado.

## 10. Requisitos de qualidade

Correção: rejeitar capacidade impossível, carga horária inválida e data inexistente. Consistência: certificado e outbox confirmam ou revertem juntos. Resiliência: falha de fila mantém pendência e duplicatas não repetem o ledger.

Manutenibilidade: domínio puro e ausência de imports entre serviços. Operabilidade: smoke do fluxo completo. Usabilidade: estados de carregamento, erros e confirmação de exclusão, com textos diretos na interface.

## 11. Riscos e pendências

Cloud e imagens Docker Hub dependem de contas ainda não confirmadas. Não há URL de produção validada. O vídeo exige fala real dos quatro alunos e publicação como não listado.

O computador de desenvolvimento não tem Docker. A integração real passou no pipeline GitHub Actions 40413543638, com CRUD, agregação, publicação e processamento de eventos. Isso comprova a execução com containers e bancos reais no runner. A demonstração cloud ainda depende da implantação pública. Prints locais e testes com memória não substituem a evidência cloud.

Polling da outbox e réplicas mínimas podem consumir recursos continuamente. Listagens têm limite de 1000 registros e ledger de 100 entidades. Indicadores cobrem esse conjunto. Paginação, autenticação institucional, migrações versionadas e rastreamento distribuído são evoluções.

## 12. Glossário e referências

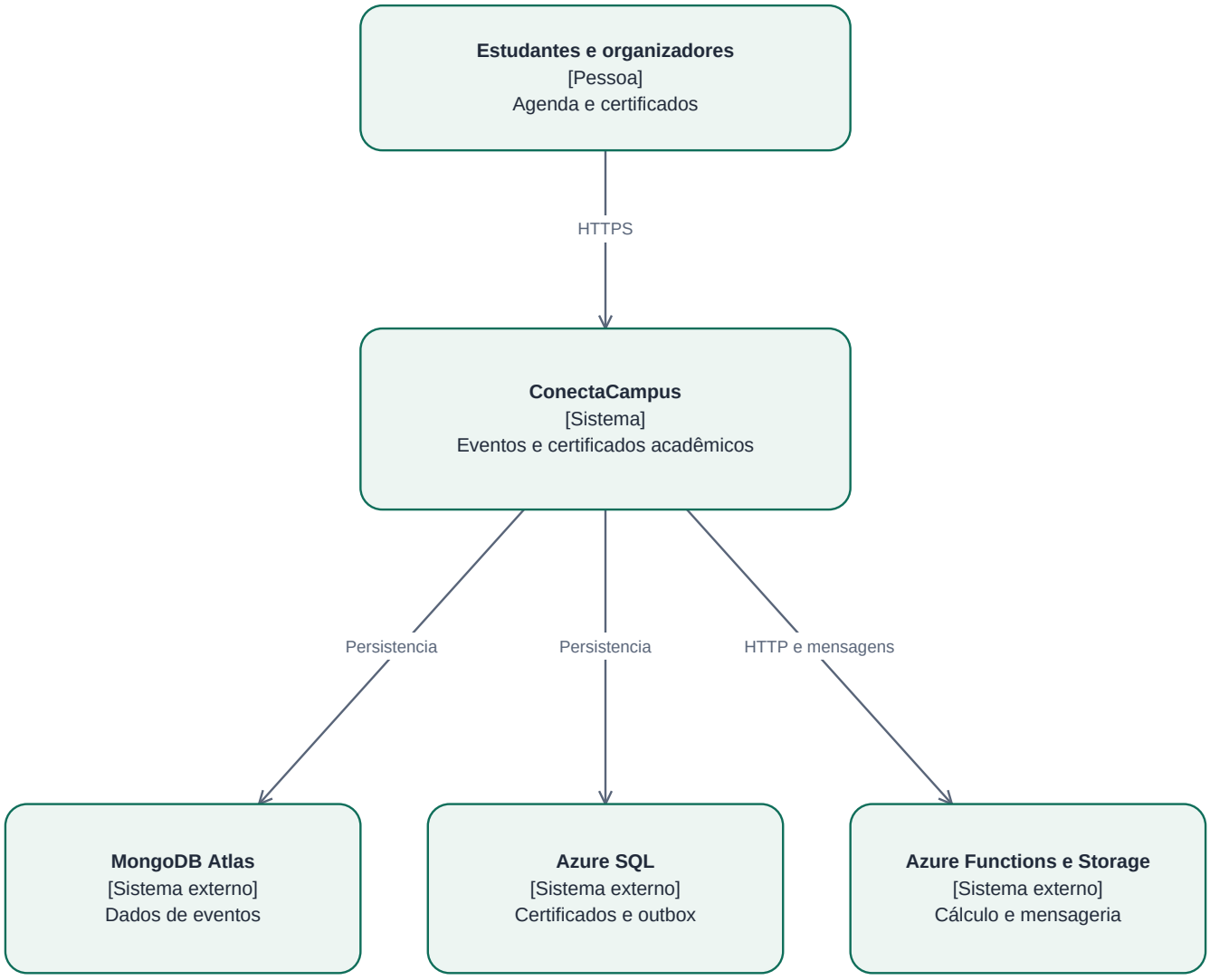
BFF: backend específico da interface. Slice: caso de uso por operação. Outbox: eventos na transação do dado de negócio. Ledger: registro idempotente. Gateway: entrada central com encaminhamento e controles.

Links verificados e pendências ficam em docs/links.json e na tabela do PDF. Cloud, Docker Hub e YouTube ficam pendentes até existirem evidências reais. O roteiro do vídeo e o guia de publicação acompanham o projeto.

Referências oficiais: https://learn.microsoft.com/azure/azure-functions/functions-reference-node e https://learn.microsoft.com/azure/azure-functions/functions-bindings-storage-queue-trigger. Oferta SQL: https://learn.microsoft.com/azure/azure-sql/database/free-offer. Ingress: https://learn.microsoft.com/azure/container-apps/ingress.

# C4: contexto

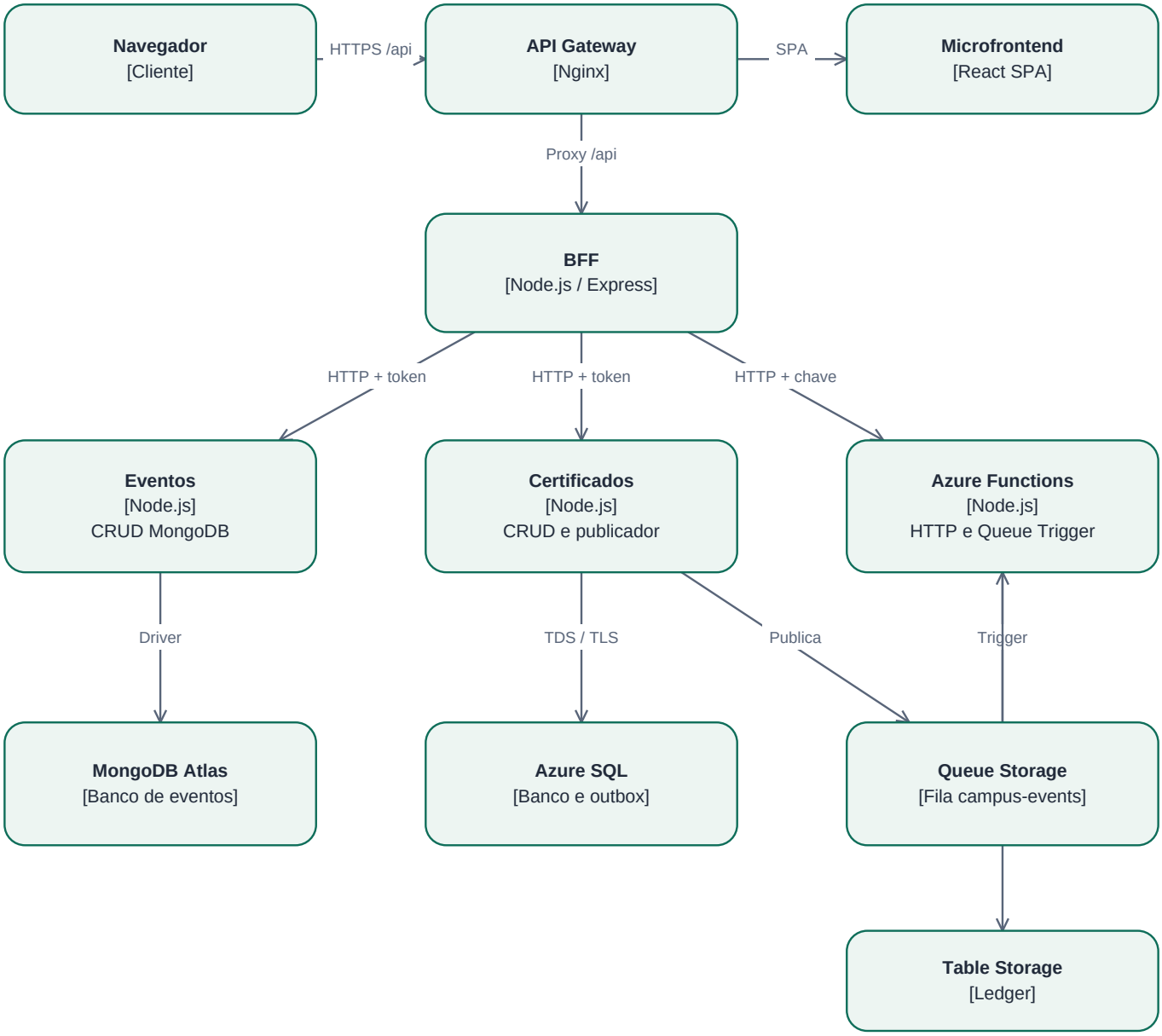
Nível 1. Pessoas, sistema e dependências externas.




Escopo funcional: eventos e certificados. A infraestrutura Azure é planejada, ainda sem URL de produção validada.

# C4: containers

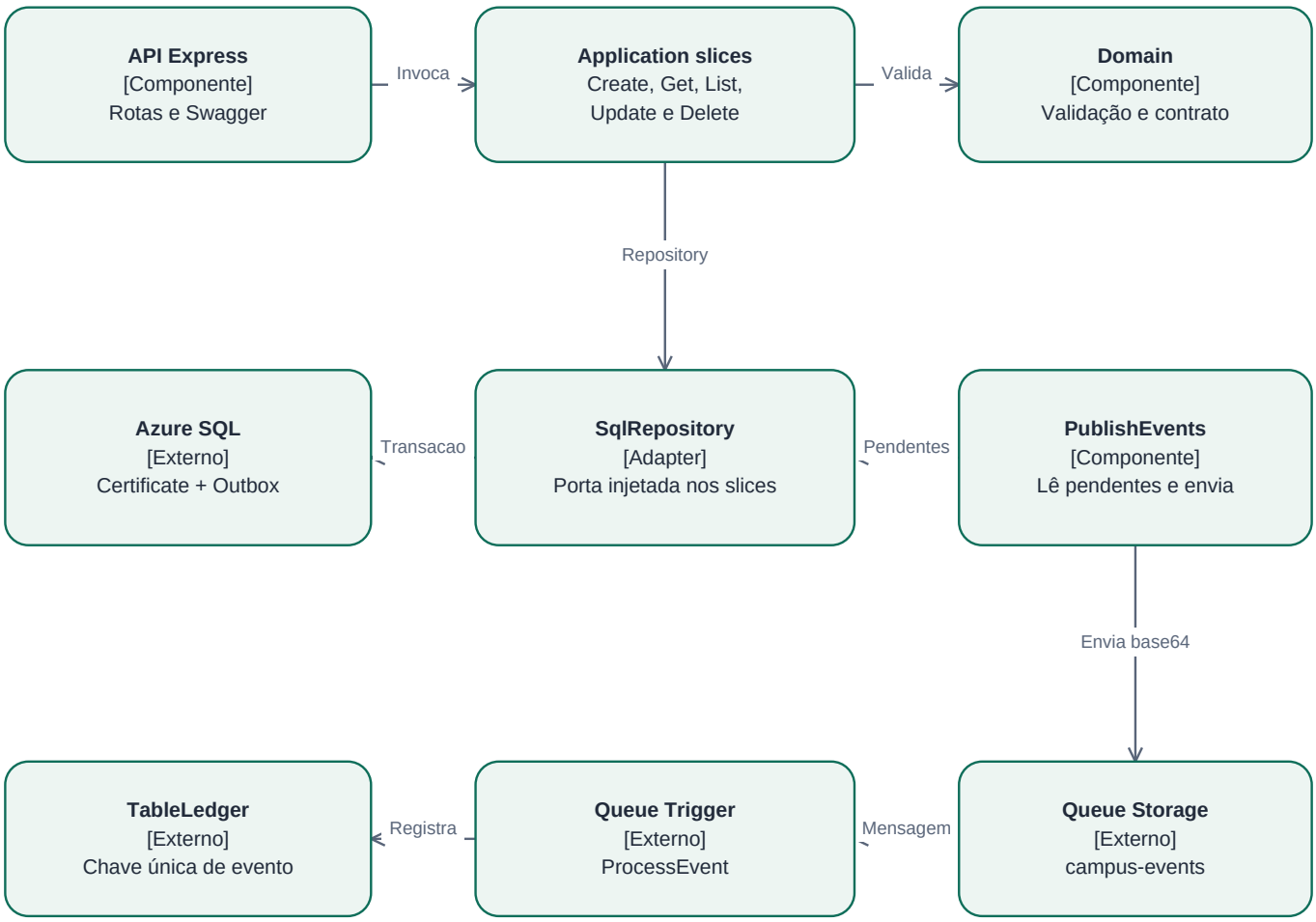
Nível 2. Chamadas síncronas e processamento assíncrono.




O código React roda no navegador e chama /api no gateway. Na nuvem, somente o gateway tem ingress público entre os containers. A Function HTTP usa chave de acesso.

# C4: componentes

Nível 3. Interior do microserviço de certificados.




O publicador confirma publishedAt depois do envio. Uma falha intermediária gera reentrega. O consumidor trata conflito de chave como duplicata. A consistência é eventual.

# Architecture Canvas

| Área | Decisão |
| --- | --- |
| Problema | Centralizar agenda acadêmica e certificados de participação |
| Pessoas | Estudantes, organizadores acadêmicos e avaliadores do PJBL |
| Valor | Consultar vagas e administrar registros de eventos e certificados |
| Domínios | Eventos no MongoDB, certificados no SQL |
| Entradas | React pelo gateway, BFF HTTP e trigger de fila |
| Integrações | MongoDB Atlas, Azure SQL, Functions, Queue e Table Storage |
| Qualidade | Invariantes no domínio, dependências voltadas para dentro, transação e idempotência |
| Implantação | Imagens independentes, gateway público, containers internos, Function protegida por chave |
| Riscos | Cotas de nuvem, cold start, segredos, eventual consistência e mensagens na poison queue |
| Evidências | Testes locais, pipeline de integração e demonstração final em URLs públicas |
| Pendências | Contas de nuvem, Docker Hub, evidências públicas e gravação do grupo |

## Links e estado da entrega

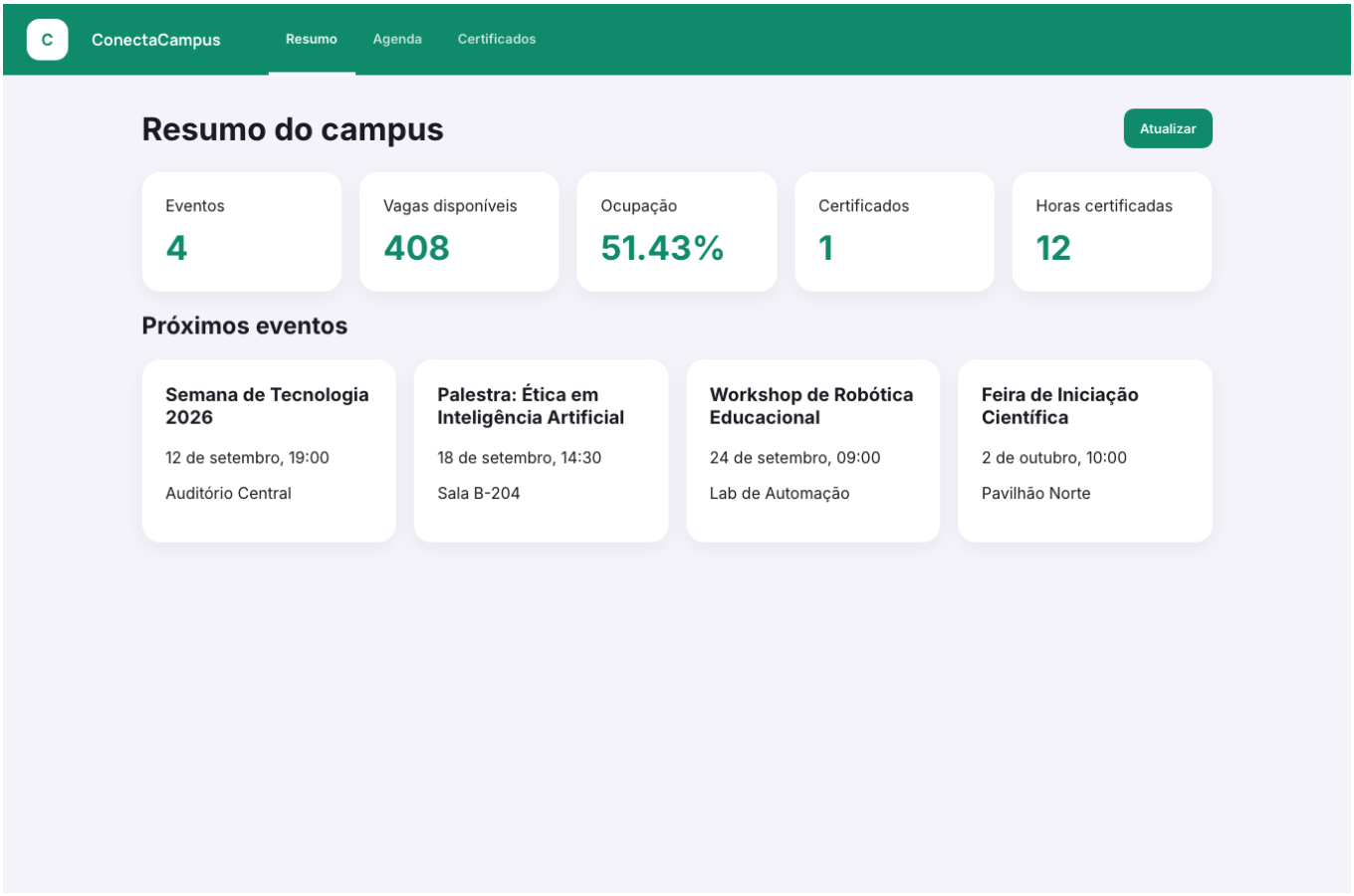
| Destino | Link ou estado |
| --- | --- |
| microfrontend | https://github.com/otavioaugustosw/conectacampus-microfrontend |
| bff | https://github.com/otavioaugustosw/conectacampus-bff |
| mongodb | https://github.com/otavioaugustosw/conectacampus-eventos |
| sql | https://github.com/otavioaugustosw/conectacampus-certificados |
| function | https://github.com/otavioaugustosw/conectacampus-functions |
| Integração | https://github.com/leosmkk/conectacampus/tree/feat/pjbl-distributed-architecture |
| Proposta de alteração | https://github.com/leosmkk/conectacampus/pull/2 |
| Pipeline | https://github.com/leosmkk/conectacampus/actions/runs/40413543638 |
| dockerHub | Pendente de publicação e validação |
| frontend | Pendente de publicação e validação |
| gateway | Pendente de publicação e validação |
| swaggerBff | Pendente de publicação e validação |
| swaggerMongo | Pendente de publicação e validação |
| swaggerSqlAndEvents | Pendente de publicação e validação |
| youtube | Pendente de publicação e validação |

## Caminhos após a publicação

No gateway público: /api/docs/ para BFF, /api/docs/?url=/api/specs/eventos para Mongo e /api/docs/?url=/api/specs/certificados para SQL e eventos. O contrato assíncrono está em docs/asyncapi.yaml.

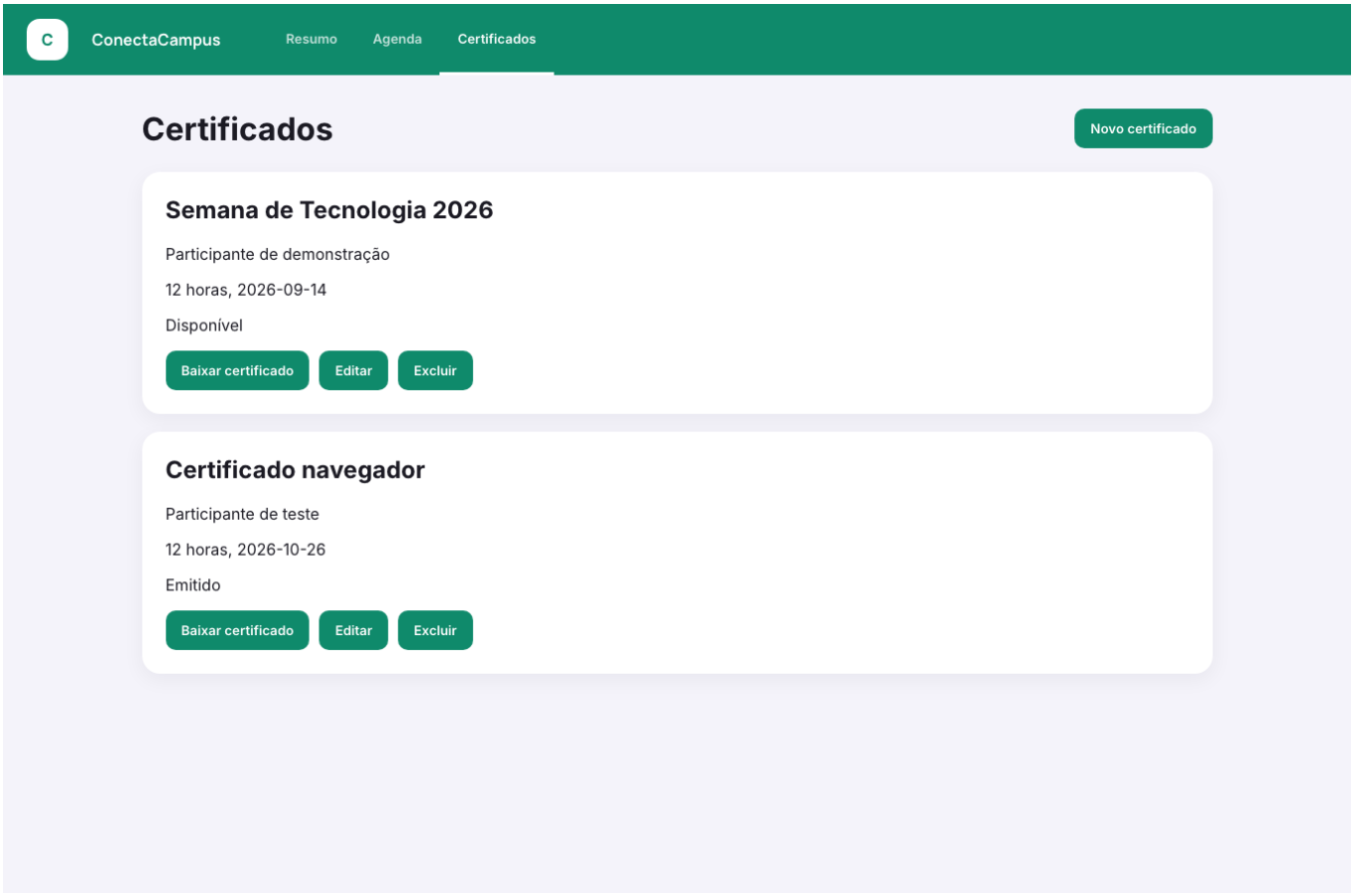
# Frontend em funcionamento

Prévia local em memória. Não comprova implantação cloud.



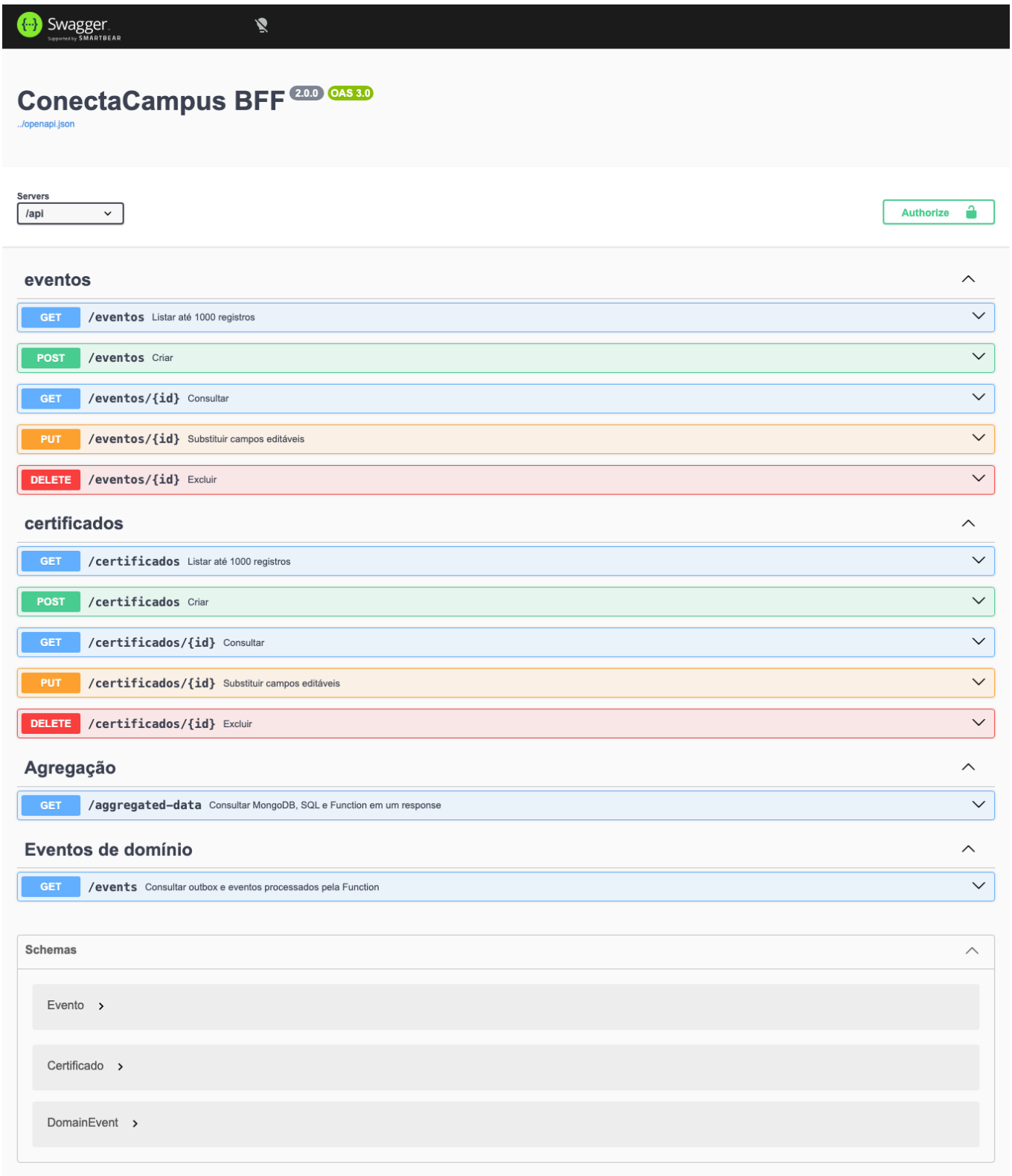
# CRUD de certificados

Prévia local após emissão. O teste de navegador verificou criação, alteração, download e exclusão.



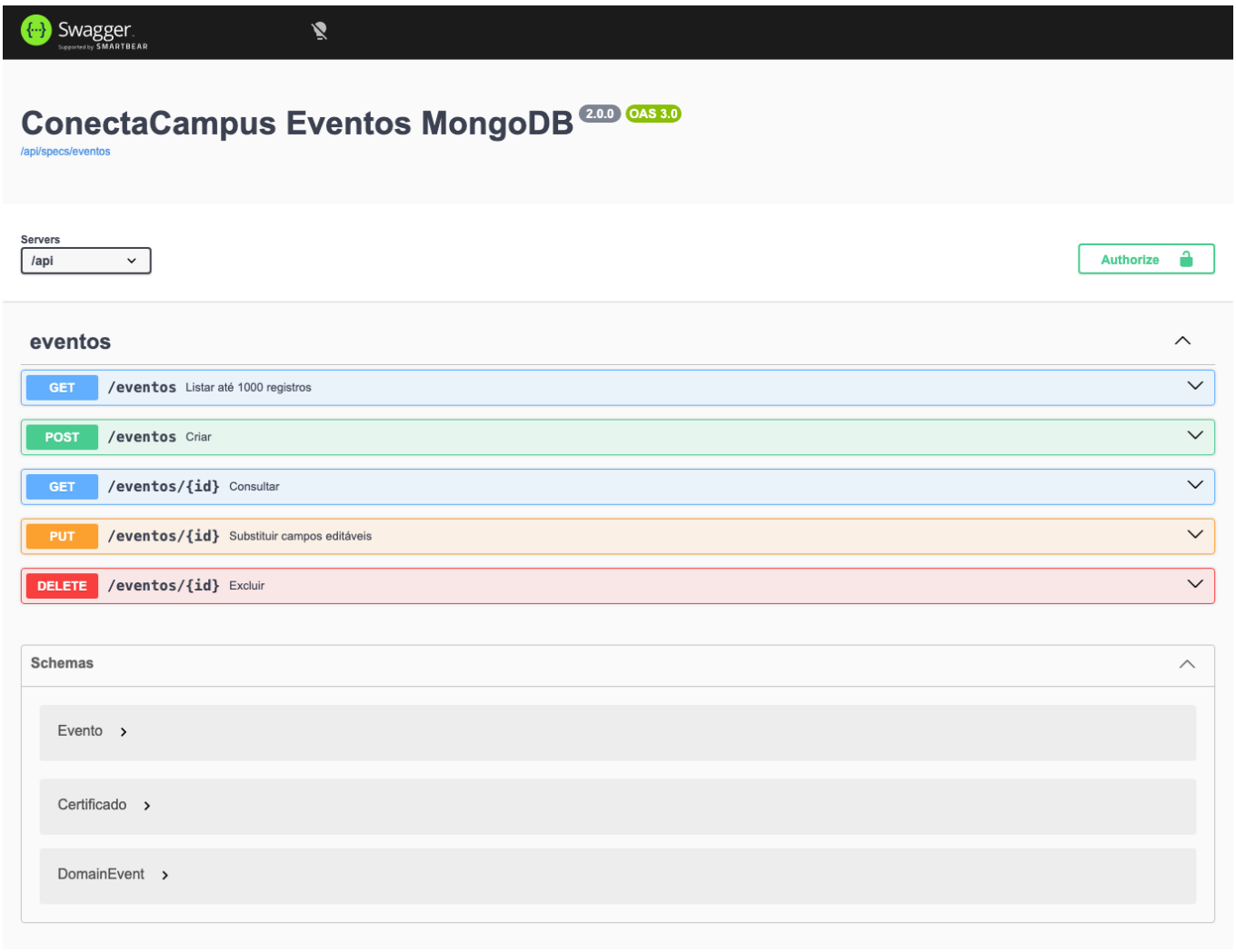
# Swagger BFF

Captura local da documentação de agregação e CRUD.



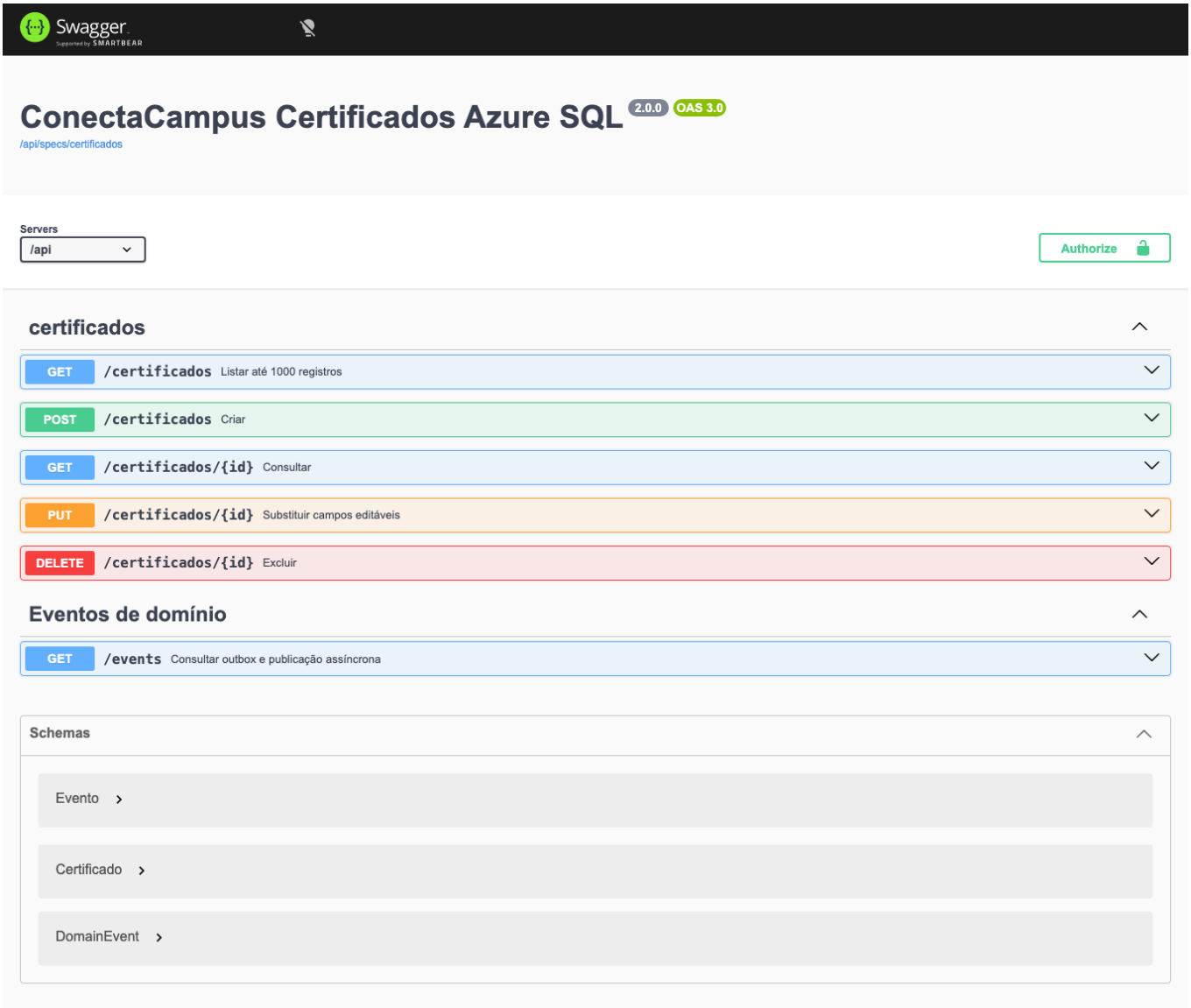
# Swagger microserviço MongoDB

Captura local pelo BFF. Integração real aprovada no pipeline GitHub Actions.



# Swagger microserviço SQL e eventos

Captura local do contrato do serviço SQL e endpoint de eventos.



## Vídeo e conclusão da entrega

Publicação pendente. O vídeo deve ser não listado e mostrar URLs cloud. Cada integrante precisa gravar sua fala real.

Publicação pendente. Cada integrante precisa gravar sua própria fala. Enviar ao YouTube como não listado e registrar o link em docs/links.json. Mostrar URLs públicas durante toda a demonstração.

1. Anna Quezia Dos Santos: apresentar o objetivo, abrir o frontend na URL cloud e mostrar Resumo e Agenda.

2. Laura Rodrigues de Souza de Camargo: criar, consultar, editar e excluir um evento. Mostrar validação de vagas e Swagger do microserviço Mongo pelo gateway.

3. Leonardo Smijtink: criar, consultar, editar e excluir um certificado. Demonstrar emissão, SQL e a outbox na mesma transação.

4. Otávio Augusto Fermino da Silva: mostrar GET /aggregated-data, os dois serviços, a chamada da Function e os eventos processados. Apresentar gateway, C4, imagens Docker Hub e testes.

Antes de gravar, rodar o smoke contra a URL pública. Durante a explicação, mostrar a aba de rede do navegador com as chamadas ao gateway/BFF, os registros de dados nos dois bancos e o processamento na fila. Não mostrar segredos nos painéis ou no terminal.

## Evidências ainda necessárias

Print do API Gateway no Azure, frontend cloud, Swagger cloud, imagens publicadas no Docker Hub e link YouTube. Atualize docs/links.json, substitua as capturas locais por evidências públicas e regenere o PDF após a verificação final.